

W1D1

Learn and Repeat

Pattern 1. “for all” and “there exists” pattern

Given an array A, if all elements of A are positive, return 1. Else return 0.

Algorithm 1. (Counting method)

```
count = 0;
for (i = 0; i < n; i++)
    if (A[i] > 0) count++;
(count == n)? true : false;
```

check the condition. If true, increment the counter. If the counter value is n, return true. Else return false.

Algorithm 2. (Test the opposite condition method)

```
for (i = 0; i < n; i++)
    if (A[i] <= 0) return false;
return true;
```

check the “opposite condition”. If true, return false. Return true outside the loop.

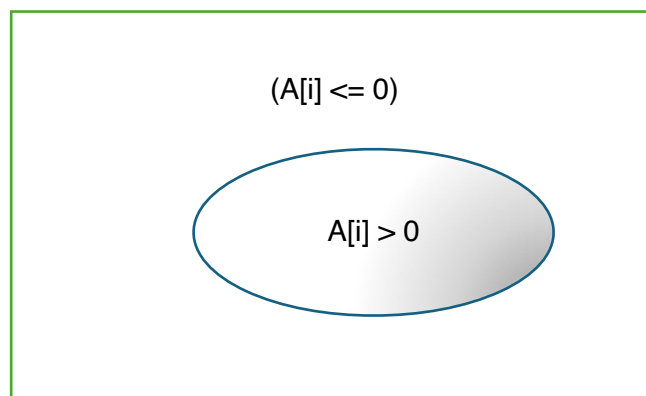
	Best case	Average case	Worst case
Algorithm 1	$O(n)$	$O(n)$	$O(n)$
Algorithm 2	$O(1)$	$O(n)$	$O(n)$

Lower bound of the problem

What is the minimum information required to conclude the problem statement in the affirmative? (This a very confusing idea for students with less exposure to logic.)

You need to check all elements of A. Hence the lower bound of the problem is $\Omega(n)$.

Since the lower bound of the problem is equal to the **worst case time complexity** in Solution 1 (Solution 2), Solution 1 (Solution 2) is optimal.



Exercise 1

An array is defined to be a **235 array** if the number of elements divisible by 2 plus the number of elements divisible by 3 plus the number of elements divisible by 5 plus the number of elements not divisible by 2, 3, or 5 is equal to the number of elements of the array.

Write a method named `is123Array` that returns 1 if its array argument is a 235 array, otherwise it returns 0.

Note : A number can be divisible by more than one number. For example, 10 is divisible by both 2 and 5.

Algorithm 3. (Counting method)

```
static int is123Array(int A[]) {  
    int n = A.length;  
    int count = 0;  
    for (int i = 0; i < n; i++)  
        if (divisorCount(A[i]) <= 1) count++;  
    return (count == n) ? 1 : 0;  
}
```

```
static int divisorCount(int x) {  
    int c = 0;  
    if (x % 2 == 0) c++;  
    if (x % 3 == 0) c++;  
    if (x % 5 == 0) c++;  
    return c;  
}
```

Algorithm 4. (Test the opposite condition method)

```
static int isNot123Array(int A[]) {  
    int n = A.length;  
    for (int i = 0; i < n; i++)  
        if (divisorCount(A[i]) >= 2) return 0;  
    return 1;  
}
```

Fill in the cells.

	Best case	Average case	Worst case
Algorithm 1	$O(n)$	$O(n)$	$O(n)$
Algorithm 2	$O(1)$	$O(n)$	$O(n)$

What is the lower bound of the problem?

$O(n)$. Solving this problem requires checking every array element. That clearly explains that is linear time or $O(n)$.